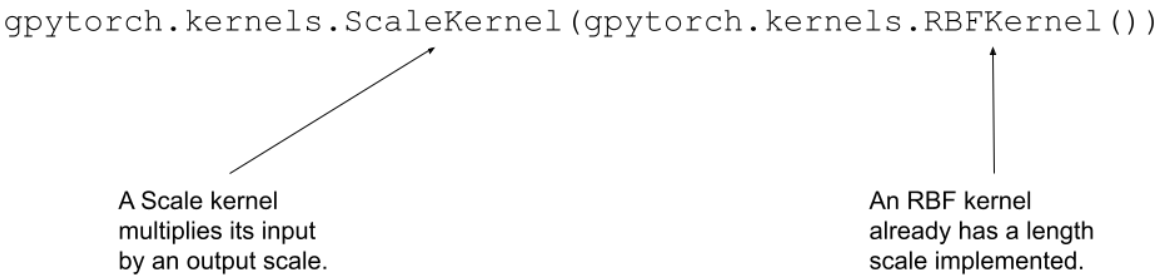
```
class ScaleGPModel(gpytorch.models.ExactGP):
    def __init__(self, train_x, train_y, likelihood):
        super().__init__(train_x, train_y, likelihood)
        self.mean_module = gpytorch.means.ZeroMean()
        self.covar_module =
        <linearrow />   gpytorch.kernels.ScaleKernel(
          gpytorch.kernels.RBFKernel())   ❶

    def forward(self, x):
        ...   ❷
```

❶ `gpytorch.kernels.ScaleKernel` implements the output scale.

❷ Omitted.

Notice that our code for the `covar_module` attribute here is different from before: we are placing a `gpytorch.kernels.ScaleKernel` object outside of our usual RBF kernel. This effectively implements the output scale that scales the output of the RBF kernel by some constant factor. The length scale, on the other hand, is already included within `gpytorch.kernels.RBFKernel`.



In the code we have been using thus far, there is one line where we set the length scale of our kernel with `model.covar_module.base_kernel.lengthscale = lengthscale`. This is where the value of the length scale is stored. Using a similar API, we can set the output scale of our kernel with `model.covar_module.outputscale = outputscale`.

Now, to see that the length scale effectively controls how fast the function varies, we compare the predictions made by two GPs, one with a length scale of 1, and the other with a length scale of 0.3, in Figure 3.11.

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>